

Implementation Guide

Cloud-Based Data Analytics Pipeline for Smart Healthcare

This guide walks through how we built the data pipeline on AWS, step by step, with a screenshot at each stage. Everything was built in the **us-east-1 (N. Virginia)** region using only AWS Free Tier services. The data flows like this: raw files go into Amazon S3, AWS Glue reads and organises them, Amazon Athena runs the analysis with SQL, and Amazon Quick Sight turns the results into charts.

1. Amazon S3 — Storing the Data

First, we made one storage bucket to hold all our files. Think of the bucket as a big folder in the cloud. Inside it we made four folders that separate the data by how processed it is: **bronze** for the raw files we upload, **silver** for cleaned data, **gold** for the final ready-to-use data, and **Athena-results** where Athena saves its query output.

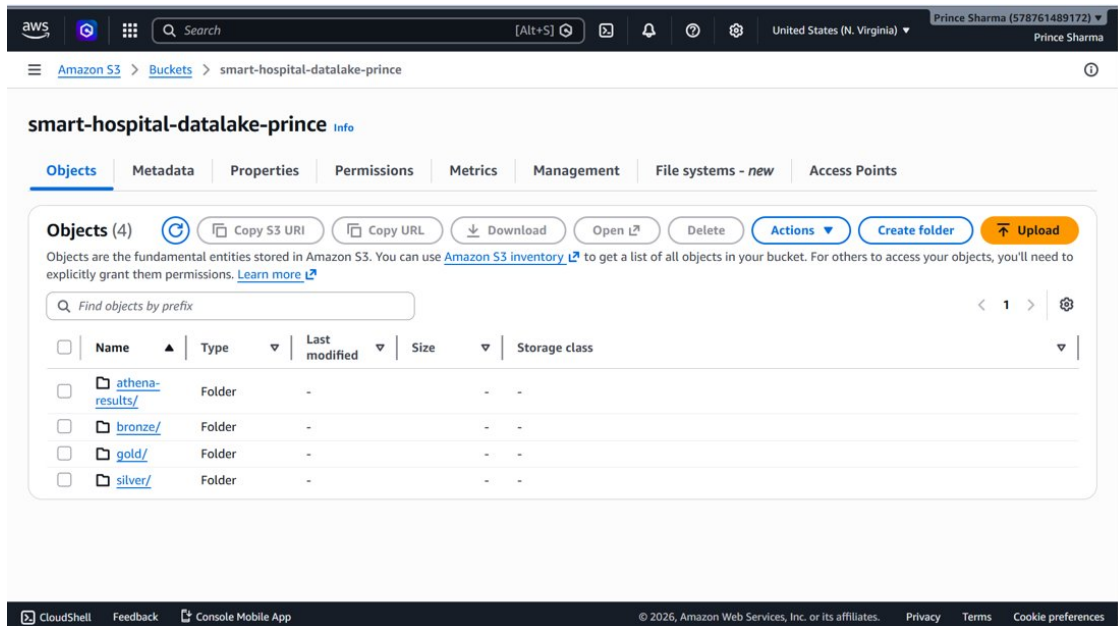


Figure 1 — The S3 bucket with its four folders: bronze, silver, gold, athena-results

Inside the **bronze** folder we made three more folders, one for each type of data. Keeping each file in its own folder is what lets AWS later detect them as three separate tables instead of mixing them together.

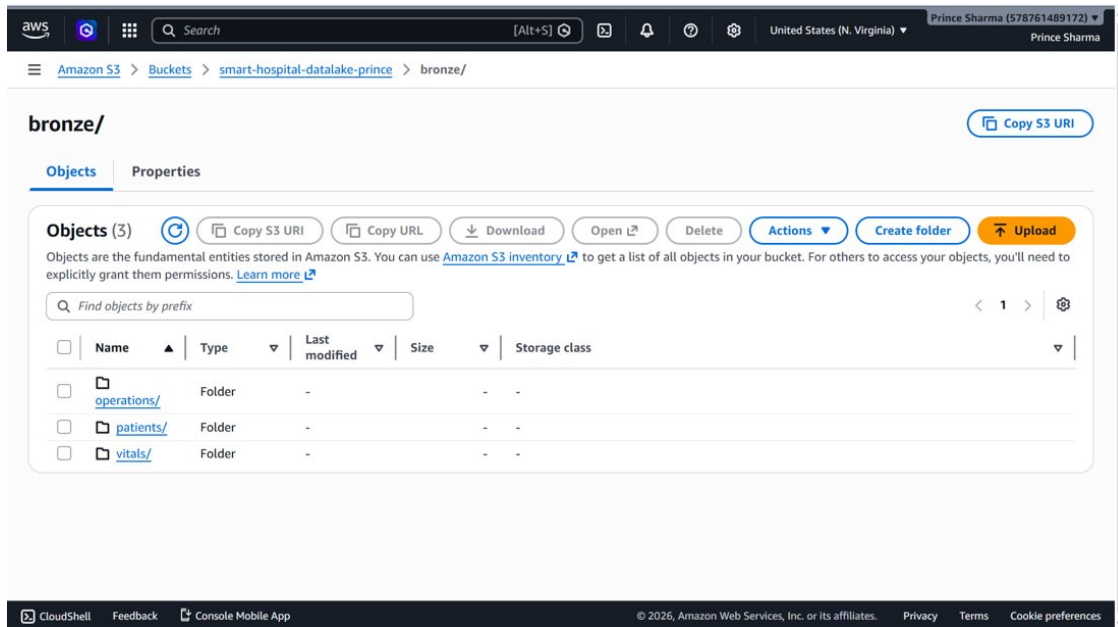


Figure 2 — Inside bronze: separate folders for operations, patients and vitals

2. IAM — Giving Permission

AWS services can touch each other data unless you allow it. So, we created a permission role called **Glue Healthcare Role** and gave it two permissions: one to use Glue and one to read our S3 files. Later, Glue uses this role to open our bucket.

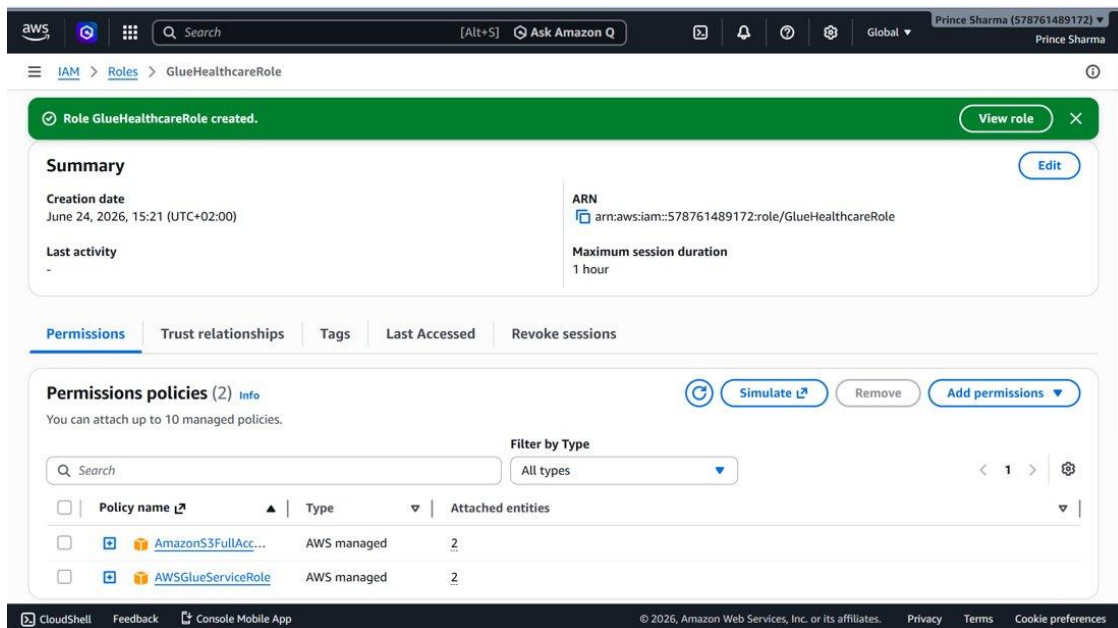


Figure 3 — The GlueHealthcareRole with its two attached permission policies

3. AWS Glue — Building the Catalog

Next, we made a database called **healthcare_db**. This is just a place to list our tables it doesn't copy the data; it only keeps track of where the data is and what columns it has.

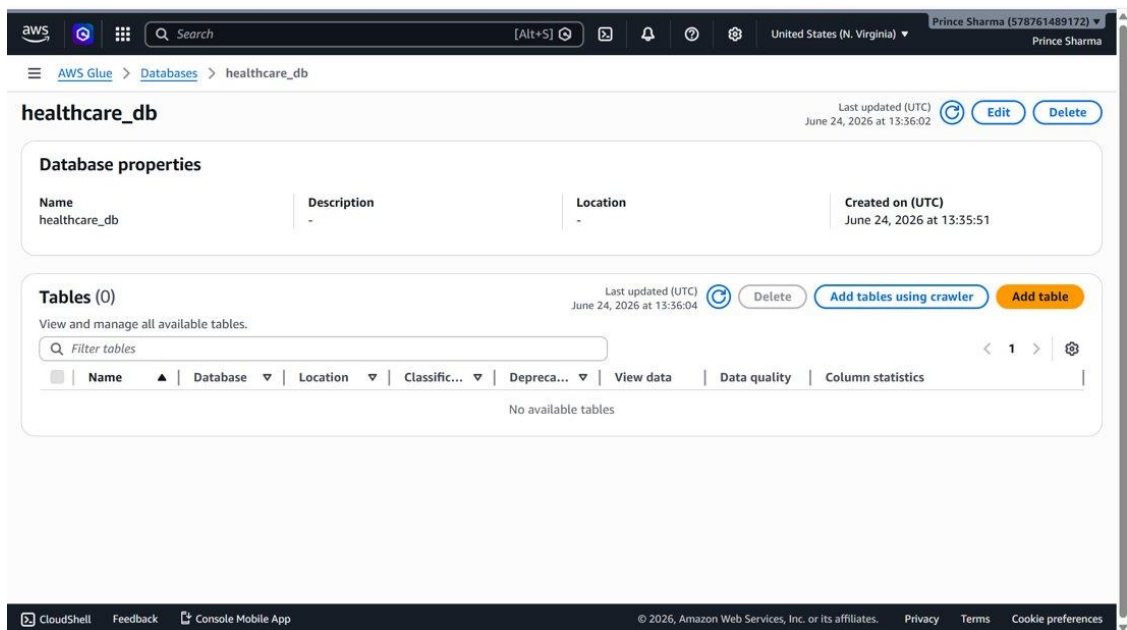


Figure 4 — The empty healthcare_db database, ready to hold our tables

Then we set up a **crawler** a small tool that looks at our bronze files and works out their structure (the column names and types) on its own. We pointed it at the bronze folder and gave it the permission role we made earlier.

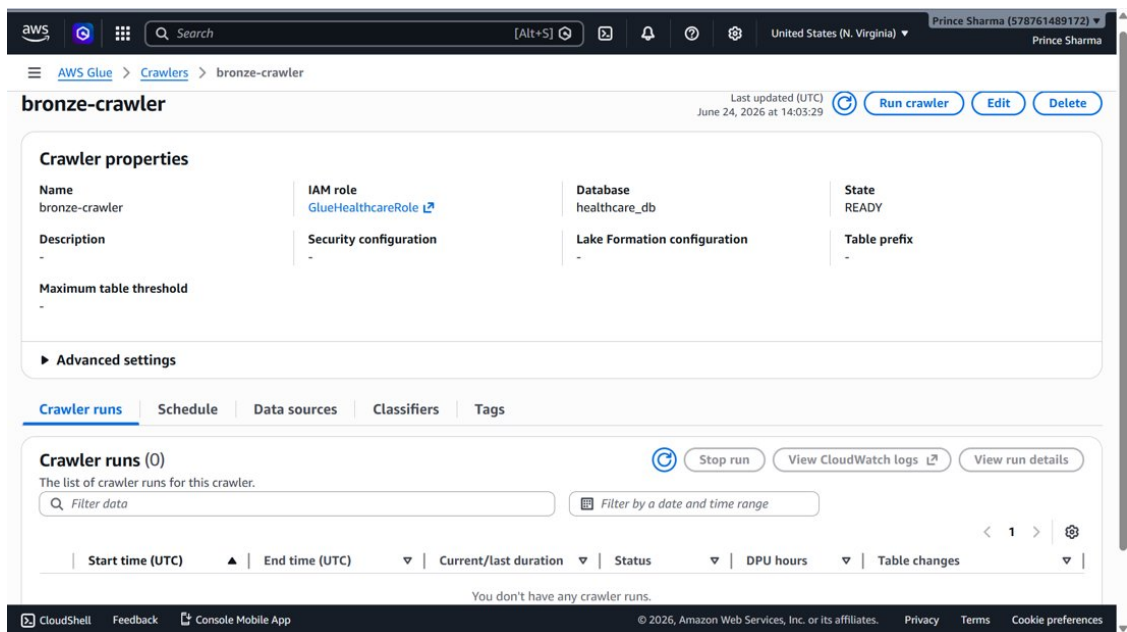


Figure 5 — The bronze-crawler settings: it uses Glue Healthcare Role and points to healthcare_db

We ran the crawler and it finished in under a minute. It read all three files and created **3 tables** automatically patients, vitals and operations with all their columns detected correctly.

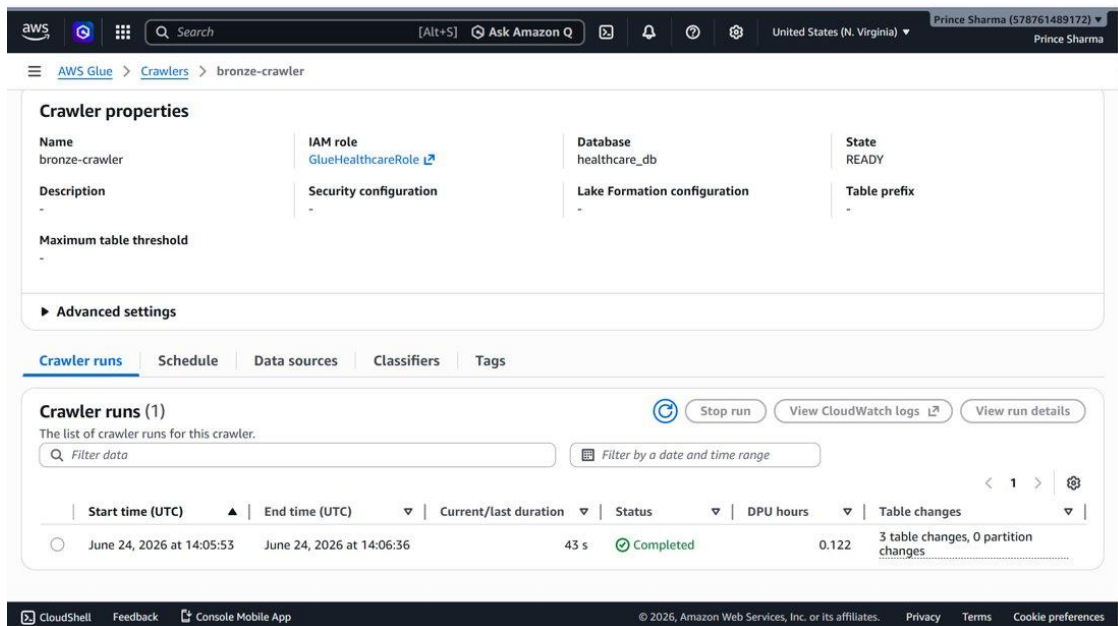


Figure 6 — The crawler run finished successfully and added 3 tables

4. Amazon Athena — Asking Questions with SQL

Now that the data was organised, we used Athena to ask questions using SQL. We picked our **healthcare_db** database and ran a quick test first to make sure everything was connected.

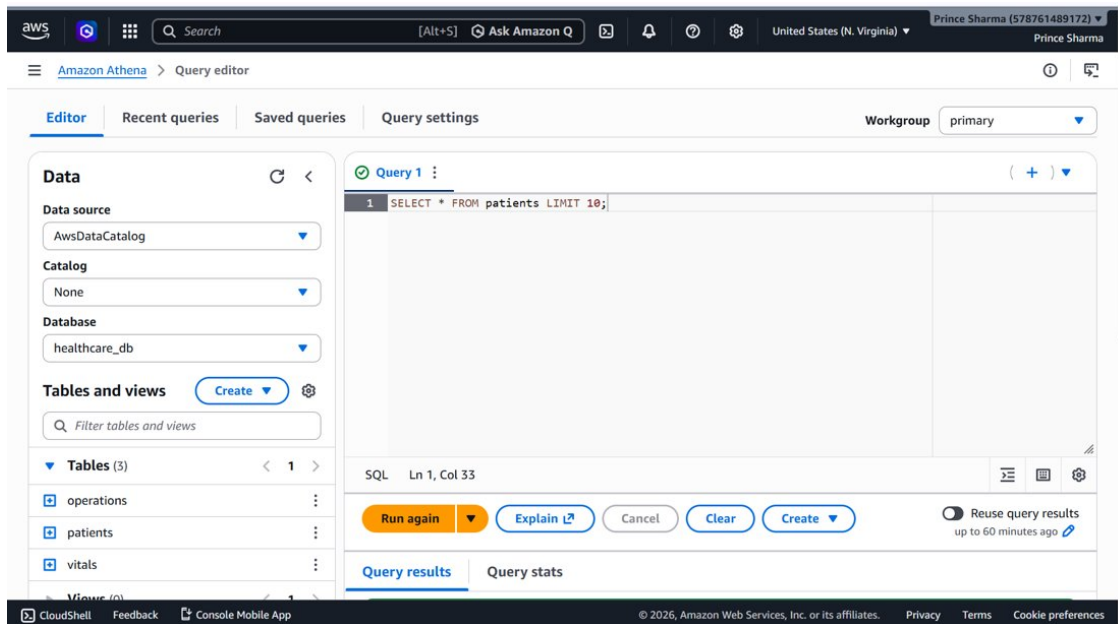
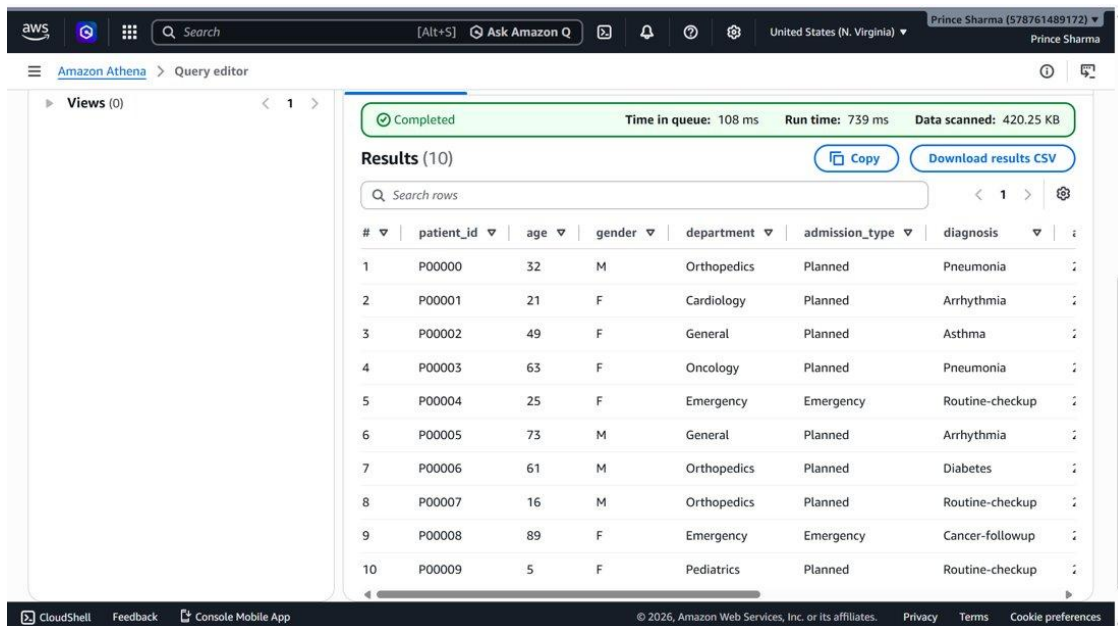


Figure 7 — The Athena editor with healthcare_db selected and a simple test query

The test worked it returned 10 patient rows, showing the data is readable and the columns (patient_id, age, gender, department, etc.) all came through correctly.



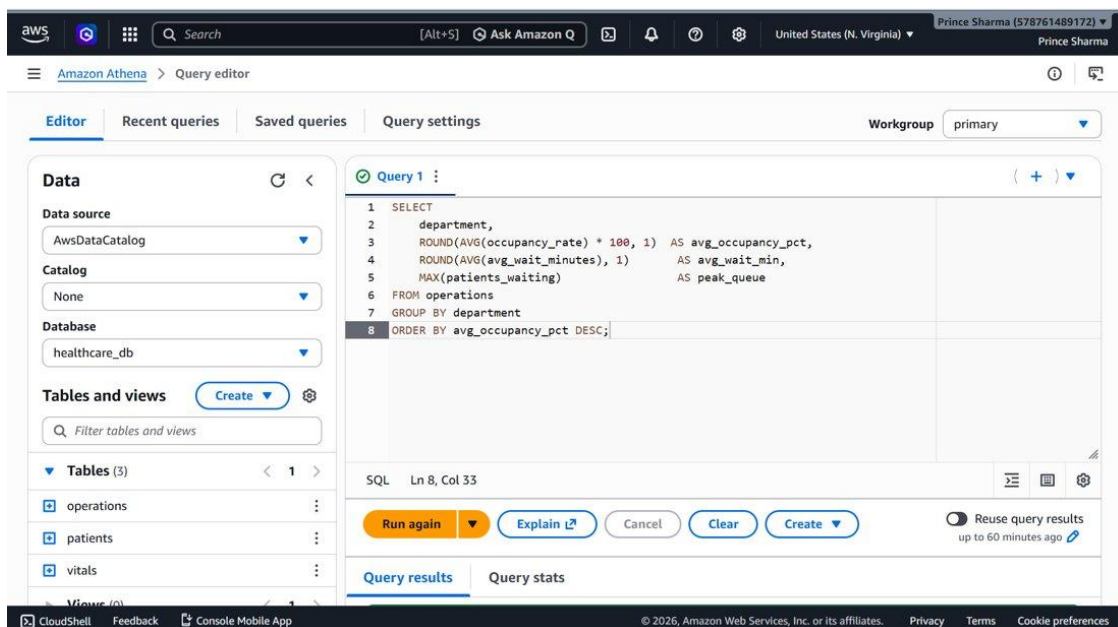
The screenshot shows the Amazon Athena Query Editor interface. At the top, there's a navigation bar with the AWS logo, a search bar, and user information for Prince Sharma. The main area is titled 'Amazon Athena > Query editor'. On the left, there's a sidebar with 'Views (0)'. The main content area shows a 'Completed' status with 'Time in queue: 108 ms', 'Run time: 739 ms', and 'Data scanned: 420.25 KB'. Below this, there's a 'Results (10)' section with a search bar and a table of 10 patient records. The table has columns: #, patient_id, age, gender, department, admission_type, and diagnosis. The records are numbered 1 through 10.

#	patient_id	age	gender	department	admission_type	diagnosis
1	P00000	32	M	Orthopedics	Planned	Pneumonia
2	P00001	21	F	Cardiology	Planned	Arrhythmia
3	P00002	49	F	General	Planned	Asthma
4	P00003	63	F	Oncology	Planned	Pneumonia
5	P00004	25	F	Emergency	Emergency	Routine-checkup
6	P00005	73	M	General	Planned	Arrhythmia
7	P00006	61	M	Orthopedics	Planned	Diabetes
8	P00007	16	M	Orthopedics	Planned	Routine-checkup
9	P00008	89	F	Emergency	Emergency	Cancer-followup
10	P00009	5	F	Pediatrics	Planned	Routine-checkup

Figure 8 — Test query result — 10 patient records returned

Query 1 — Which department is the busiest?

This query works out the average bed occupancy and waiting time for each department.



The screenshot shows the Amazon Athena Query Editor interface. The 'Editor' tab is active. On the left, there's a sidebar with 'Data' and 'Tables and views'. The 'Data' section shows 'Data source: AwsDataCatalog', 'Catalog: None', and 'Database: healthcare_db'. The 'Tables and views' section shows a list of tables: operations, patients, and vitals. The main content area shows a SQL query for 'Query 1'. The query is: SELECT department, ROUND(AVG(occupancy_rate) * 100, 1) AS avg_occupancy_pct, ROUND(AVG(avg_wait_minutes), 1) AS avg_wait_min, MAX(patients_waiting) AS peak_queue FROM operations GROUP BY department ORDER BY avg_occupancy_pct DESC; The query is highlighted in blue. Below the query, there's a status bar showing 'SQL Ln 8, Col 33'. At the bottom, there are buttons for 'Run again', 'Explain', 'Cancel', 'Clear', and 'Create'. There's also a checkbox for 'Reuse query results up to 60 minutes ago'.

```
1 SELECT
2   department,
3   ROUND(AVG(occupancy_rate) * 100, 1) AS avg_occupancy_pct,
4   ROUND(AVG(avg_wait_minutes), 1) AS avg_wait_min,
5   MAX(patients_waiting) AS peak_queue
6 FROM operations
7 GROUP BY department
8 ORDER BY avg_occupancy_pct DESC;
```

Figure 9 — The SQL for the occupancy and wait-time query

The result shows **Emergency is the busiest** at about **78.5%** occupancy, while General is the quietest at 48.9%. Emergency clearly has the most pressure on it.

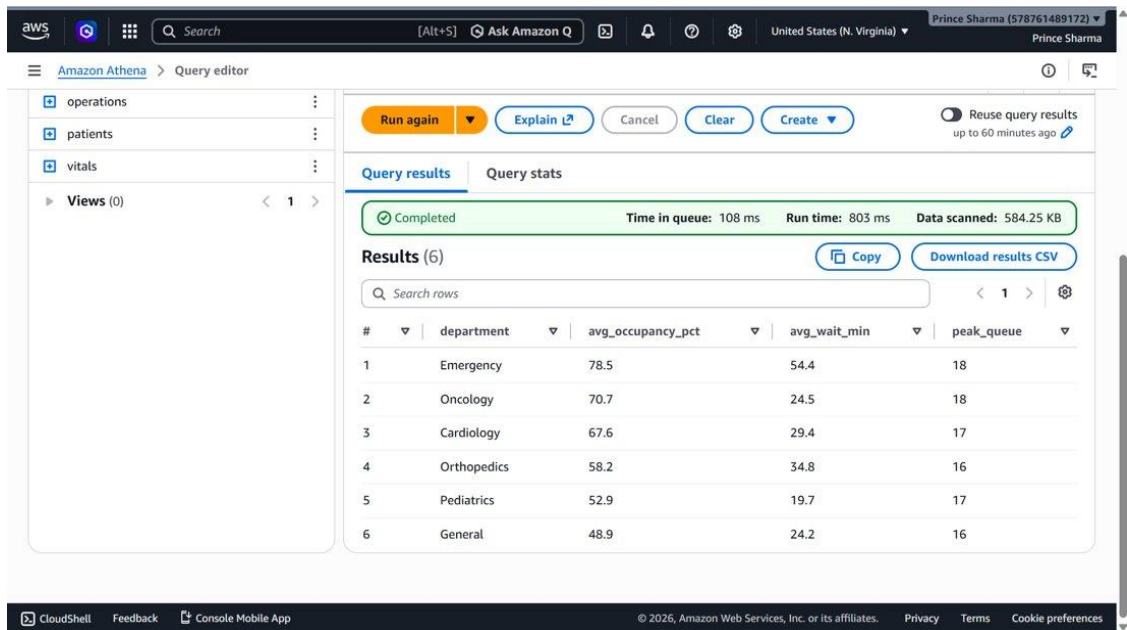


Figure 10 — Result — Emergency tops occupancy at 78.5%, General lowest at 48.9%

Query 2 — Who comes back to hospital the most?

This query checks the readmission rate (patients returning within 30 days) split by department and age group.

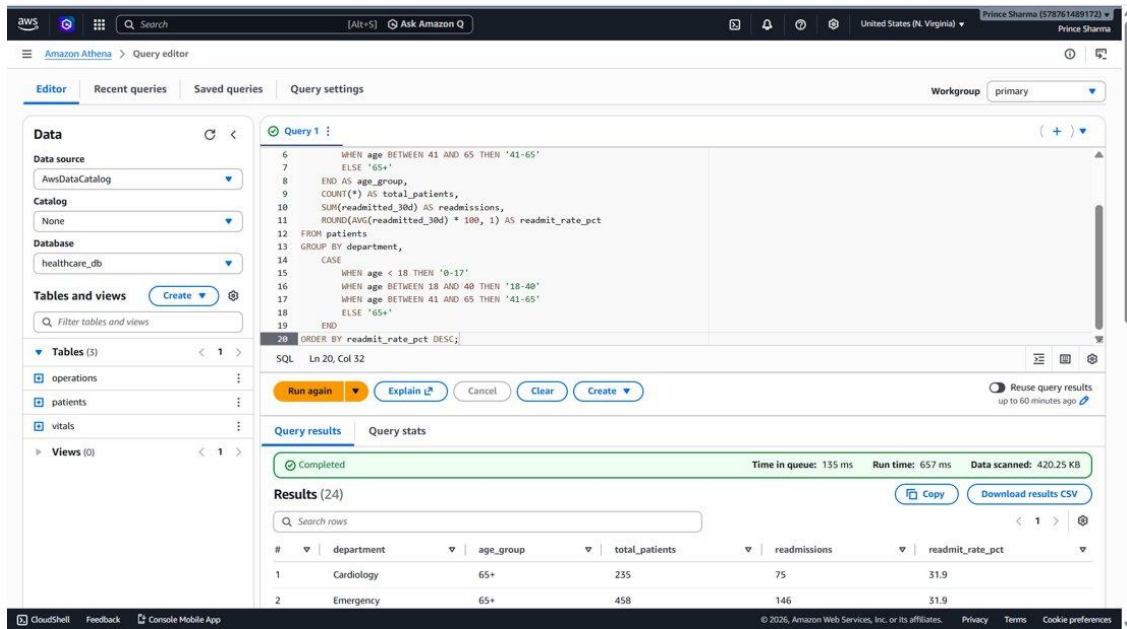


Figure 11 — The SQL for the readmission-rate query, with age groups

The result is clear: **older patients (65+) come back the most**. Cardiology and Emergency patients aged 65+ have the highest rate at about **31.9%**, while younger groups are much lower (Pediatrics 18-40 is only 7.3%).

The screenshot shows the Amazon Athena query results interface. The query has completed, with a time in queue of 135 ms and a run time of 657 ms. The results are displayed in a table with 24 rows and 6 columns: #, department, age_group, total_patients, readmissions, and readmit_rate_pct. The data shows that the highest readmission rates are for Cardiology and Emergency departments in the 65+ age group, with rates of 31.9% and 31.9% respectively.

#	department	age_group	total_patients	readmissions	readmit_rate_pct
1	Cardiology	65+	235	75	31.9
2	Emergency	65+	458	146	31.9
3	Oncology	65+	168	49	29.2
4	Oncology	41-65	145	36	24.8
5	Emergency	0-17	253	59	23.3
6	Emergency	18-40	345	77	22.3
7	Oncology	18-40	161	34	21.1
8	Emergency	41-65	386	81	21.0
9	Pediatrics	65+	221	46	20.8
10	Cardiology	18-40	169	35	20.7
11	General	65+	231	52	20.7
12	Cardiology	41-65	227	46	20.3
13	Cardiology	0-17	156	30	19.4
14	Orthopedics	65+	183	34	18.6
15	General	41-65	206	36	17.5
16	General	18-40	189	29	15.3
17	Oncology	0-17	113	17	15.0
18	General	0-17	149	22	14.8
19	Orthopedics	41-65	192	27	14.1
20	Orthopedics	0-17	129	17	13.2
21	Pediatrics	0-17	132	17	12.9
22	Orthopedics	18-40	173	19	11.0
23	Pediatrics	41-65	182	18	9.9
24	Pediatrics	18-40	178	13	7.3

Figure 12 — Result — 65+ age groups have the highest readmission rates

Query 3 — Do the IoT sensors show a difference?

This query joins the patient data with the IoT vital-sign readings to compare emergency vs planned admissions.

The screenshot shows the Amazon Athena query editor interface. The query is named 'Query 1' and is written in SQL. The query joins the 'patients' table with the 'vitals' table on the 'patient_id' column. The query calculates the average heart rate, average temperature, and average systolic blood pressure for each admission type. The results are displayed in a table with 2 rows and 8 columns: #, admission_type, patients, avg_heart_rate, avg_spo2, avg_temp_c, avg_systolic_bp. The results show that the average heart rate is higher for emergency admissions (105.1) compared to planned admissions (77.9).

```

1 SELECT
2   p.admission_type,
3   COUNT(DISTINCT p.patient_id) AS patients,
4   ROUND(AVG(v.heart_rate), 1) AS avg_heart_rate,
5   ROUND(AVG(v.spo2), 1) AS avg_spo2,
6   ROUND(AVG(v.temperature), 2) AS avg_temp_c,
7   ROUND(AVG(v.bp_systolic), 0) AS avg_systolic_bp
8 FROM vitals v
9 JOIN patients p
10  ON v.patient_id = p.patient_id
11 GROUP BY p.admission_type
12 ORDER BY avg_heart_rate DESC

```

#	admission_type	patients	avg_heart_rate	avg_spo2	avg_temp_c	avg_systolic_bp
1	Emergency	1868	105.1	93.0	37.6	139.0
2	Planned	3132	77.9	96.9	36.8	119.0

Figure 13 — The SQL joining vitals with patients by admission type

Query 4 — How does occupancy change day by day?

This last query groups the data by day so we can see the trend over the three months. It returned 90 days of data, one row per day, ready to plot as a line.

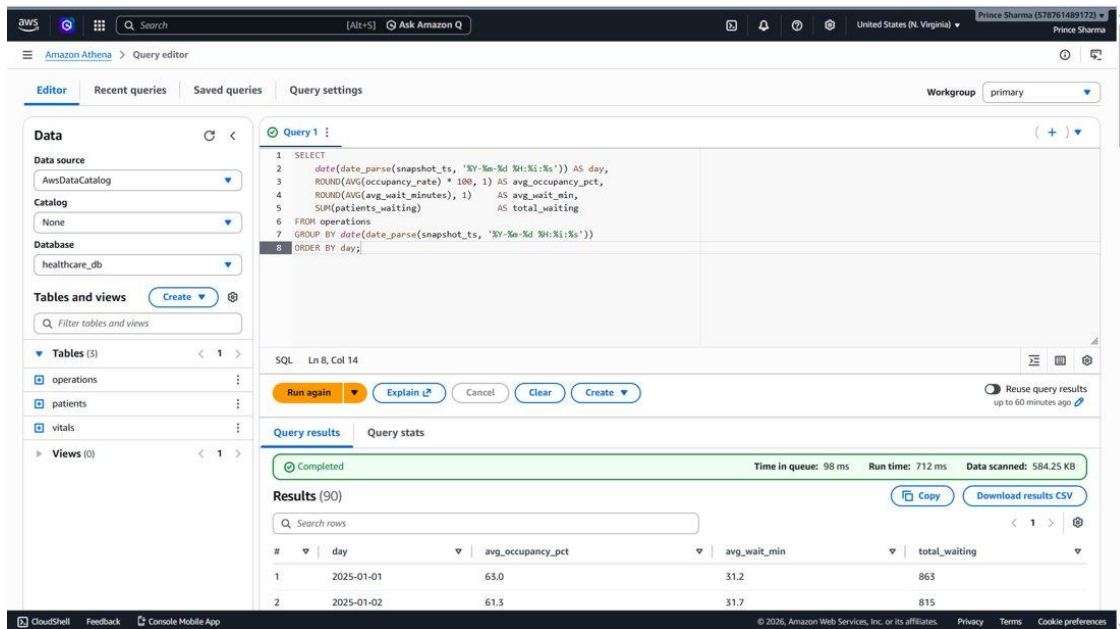


Figure 14 — The daily-trend SQL and its 90-day result

5. Amazon Quick Sight — Showing the Results as Charts

Finally, we connected Quick Sight to Athena and built four charts on one dashboard. This makes the numbers easy to read briefly for a hospital manager.



Figure 15 — The finished dashboard with all four charts

The four charts tell the whole story together: the **occupancy** and **wait-time** charts both show Emergency at the top, so the main bottleneck; the **readmission** chart shows risk rising with age (65+ highest); and the **daily trend** line shows occupancy staying steady around 66% with small daily ups and downs.

6. AWS Glue ETL — Cleaning and Converting the Data

So far, the crawler only catalogued the raw data. Next, we ran a Glue **ETL job** that transforms it. The job reads the raw CSVs from Bronze, cleans them (fixes data types, removes duplicates, parses dates), and saves the result as **Parquet** a compact, fast format into the **Silver** zone. It also builds three small ready-to-use summary tables into the **Gold** zone.

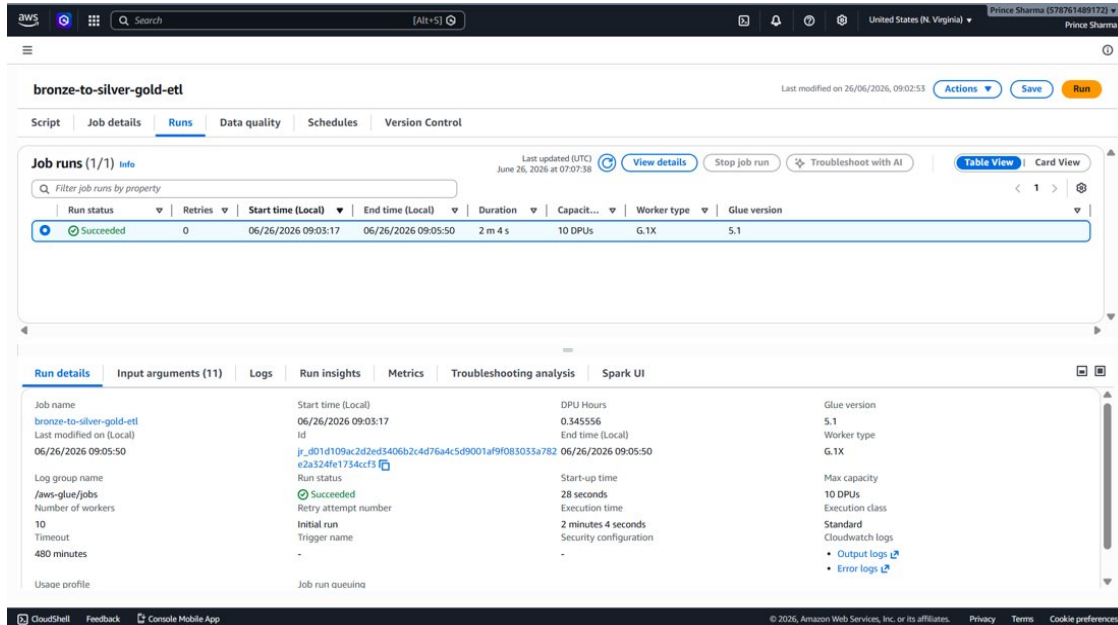


Figure 16 — The Glue ETL job finished — status Succeeded in about 2 minutes

After the job ran, the **Silver** zone filled up with cleaned Parquet files. Here is the patients data, now stored as a Parquet file instead of raw CSV.

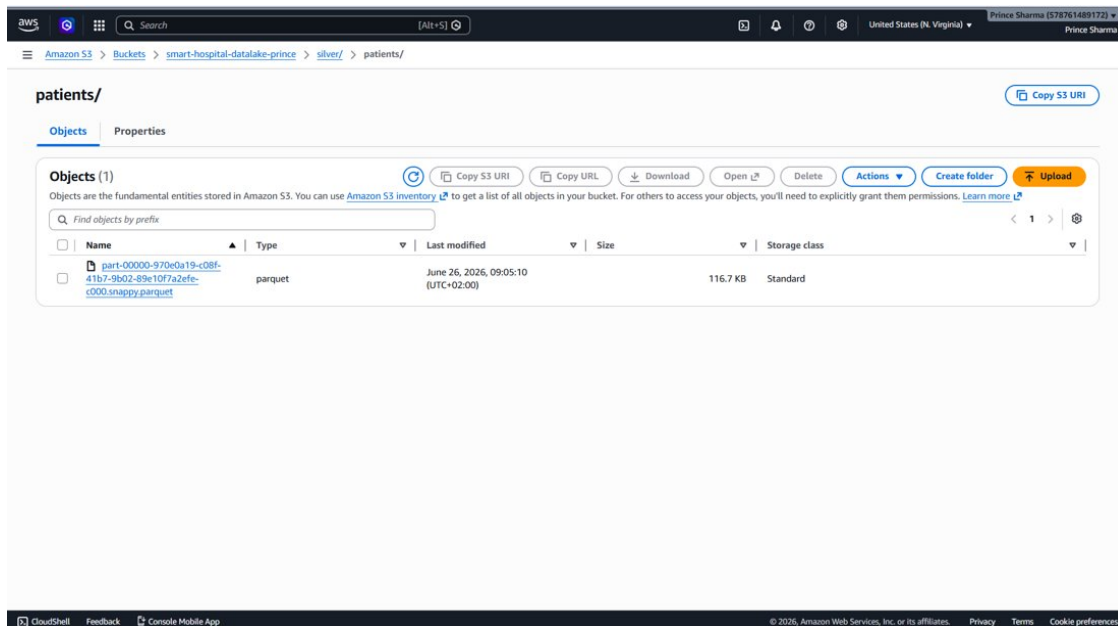


Figure 17 — Silver zone — cleaned patients data saved as Parquet

The **Gold** zone also filled with the curated summary tables (department KPIs, readmission by age, daily trend) these are the analytics-ready outputs.

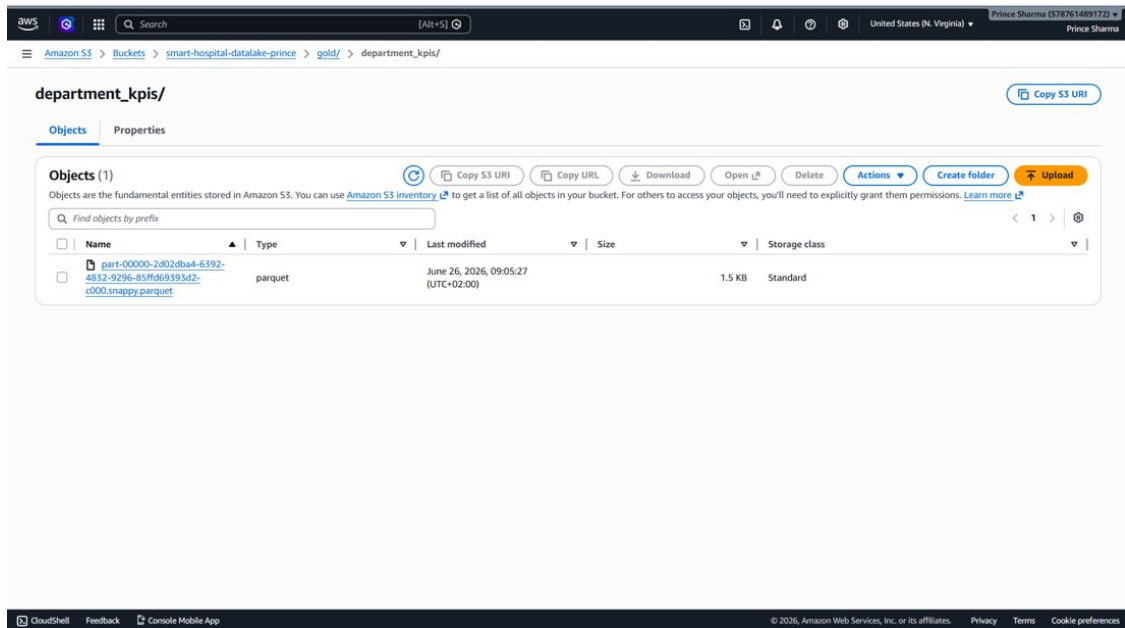


Figure 18 — Gold zone — curated department_kpis table saved as Parquet

7. Step Functions — Automating the Pipeline

Instead of running the crawler and the ETL job by hand every time, we built a **Step Functions** workflow that does both automatically, one after the other. It starts the crawler, waits for it to finish, then runs the ETL job all from a single click. This is the serverless automation the project asks for.

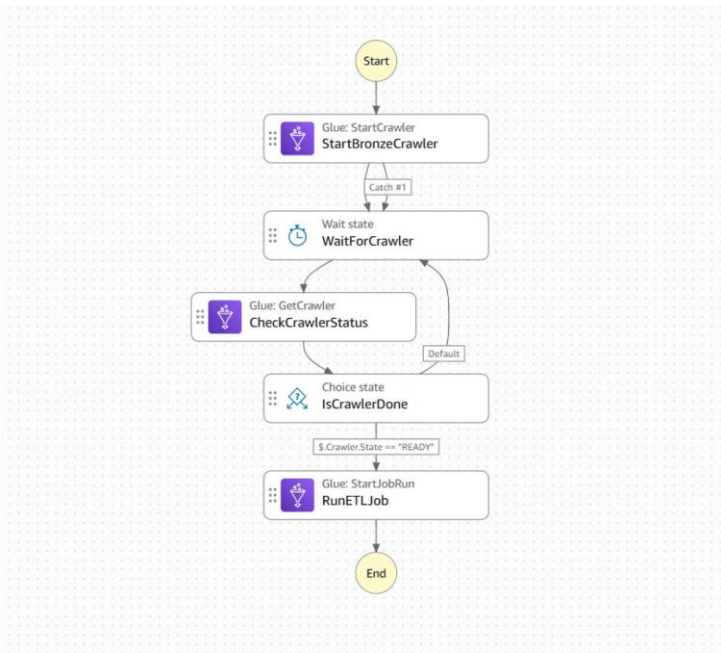


Figure 19 — The Step Functions workflow: crawler → wait → ETL job, all run automatically

8. Kinesis Data Firehose — Real-Time Ingestion

The pipeline so far loads data in batches. To show **real-time** ingestion as well, we set up a **Kinesis Data Firehose** stream. Live patient-monitoring records sent to the stream are automatically delivered into S3 within about a minute, organised into date folders. This proves the pipeline can handle streaming IoT data, not just batch files.

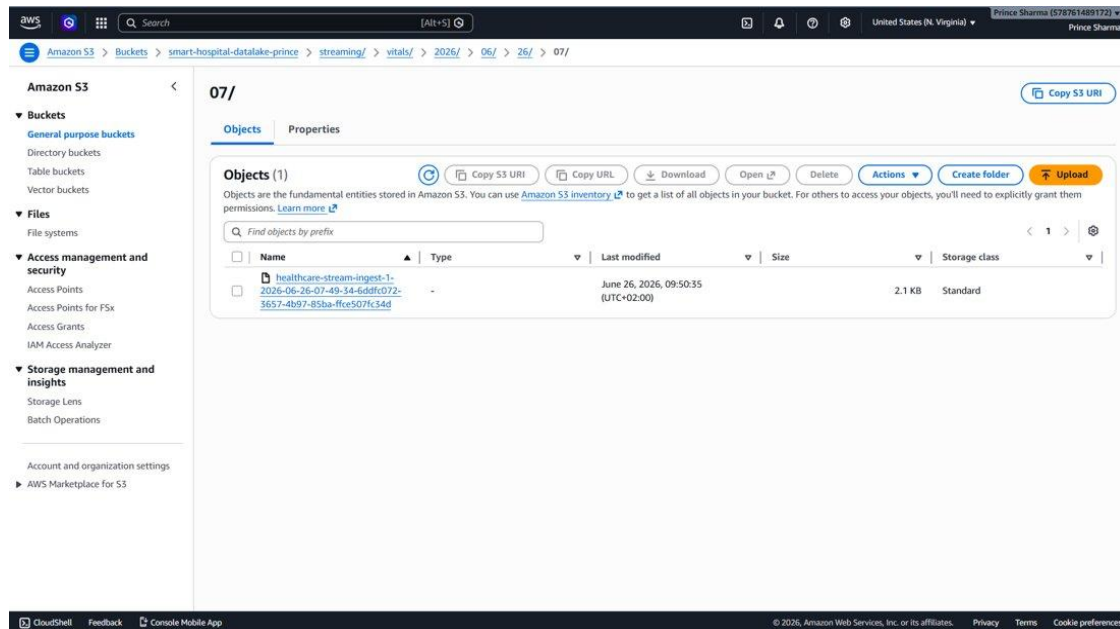


Figure 20 — Streamed vitals data delivered by Kinesis into S3 (streaming/vitals/year/month/day)

9. Result

With these steps we built a complete, working pipeline that takes hospital data in two ways (batch files and real-time streams), organises it into Bronze/Silver/Gold zones, cleans and converts it with a Glue ETL job, runs the whole flow automatically with Step Functions, analyses it with Athena, and shows clear insights in a Quick Sight dashboard all serverless and inside the AWS Free Tier. The main findings: Emergency is the busiest and slowest department, and elderly patients are the most likely to be readmitted, which makes them the key group to focus on.